

SMART CONTRACT AUDIT REPORT

for

LIQD Staking

Prepared By: Xiaomi Huang

PeckShield

June 20, 2025

Document Properties

Client	LiquidLaunch
Title	Smart Contract Audit Report
Target	LIQD Staking
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	June 20, 2025	Xuxian Jiang	Final Release
1.0-rc1	June 18, 2025	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction **4**

1.1 About LIQD Staking 4

1.2 About PeckShield 5

1.3 Methodology 5

1.4 Disclaimer 7

2 Findings **9**

2.1 Summary 9

2.2 Key Findings 10

3 Detailed Results **11**

3.1 Simplified getAccountInfo() Logic in LIQDStaking 11

3.2 Revisited _syncReward() Logic in LIQDStaking 13

4 Conclusion **15**

References **16**

PeckShield

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the LIQD Staking protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts is well implemented with extensive documentation. This document outlines our audit results.

1.1 About LIQD Staking

LIQD is an ERC20 fair launchpad and multi-market DEX aggregator on the HyperEVM. It is designed to allow users to launch tokens and trade them while also returning 100% of revenue to token holders through the LIQD Staking system. Through LIQD Staking, users can safely stake their LIQD tokens and receive proportional emissions of HYPE. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of LIQD Staking

Item	Description
Name	LiquidLaunch
Type	Ethereum Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	June 20, 2025

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- https://github.com/LIQD-Labs/liquid-staking.git (43fc255)

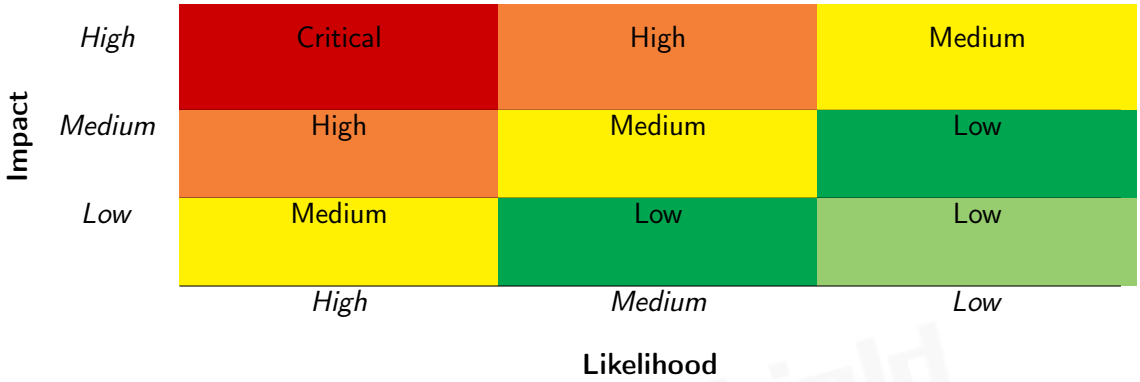
And this is the commit ID as well as current deployment address after all fixes for the issues found in the audit have been checked in.

- https://github.com/LIQD-Labs/liquid-staking.git (43fc255)
- LIQDStaking: https://www.hyperscan.com/address/0x27a9760F866DCdc655eD117c85D5592f8b4CDD1B

1.2 About PeckShield

PeckShield Inc. [5] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification



1.3 Methodology

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a checklist of items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract

Table 1.3: The Full Audit Checklist

Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [3], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings. Moreover, in case there is an issue that may affect an active protocol that has been deployed, the public version of this report may omit such issue, but will be amended with full details right after the affected protocol is upgraded with respective fixes.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logic	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 Findings

2.1 Summary

Here is a summary of our findings after analyzing the implementation of the `LIQD Staking` protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logic, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	0	
Low	0	
Informational	2	■■
Total	2	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 2 informational recommendations.

Table 2.1: Key LIQD Staking Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Informational	Simplified getAccountInfo() Logic in LIQDStaking	Coding Practices	Resolved
PVE-002	Informational	Revisited _syncReward() Logic in LIQDStaking	Coding Practices	Resolved

Besides the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to Section 3 for details.

PeckShield

3 Detailed Results

3.1 Simplified getAccountInfo() Logic in LIQDStaking

- ID: PVE-001
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: `LIQDStaking`
- Category: Coding Practices [2]
- CWE subcategory: CWE-1126 [1]

Description

In the audited `LIQDStaking` contract, there is a getter function `getAccountInfo()` that is used to retrieve comprehensive account information in a single call. Our analysis shows its logic can be simplified.

To elaborate, we show below the code snippet of the related `getAccountInfo()` routine. The above simplification refers to current approach to compute the `remainingLockTime` state, which is the remaining time in seconds until the withdrawal is available. It returns `0` if there is no pending withdrawal or the withdrawal is ready. With that, we can simplify the `if`-condition (line 573) as `if (isWithdrawalClaimable)` since `isWithdrawalClaimable` already marks the status on whether the withdrawal can now be claimed.

```
146     function getAccountInfo(
147         address account
148     )
149         external
150         view
151         returns (
152             uint256 stakedTokens,
153             uint256 currentRewards,
154             uint256 pendingWithdrawalAmount,
155             uint256 withdrawalUnlockTime,
156             bool isWithdrawalClaimable,
157             uint256 totalLifetimeRewards,
158             uint256 remainingLockTime
159         )
160     {
```

```
161         stakedTokens = stakedAmount[account];
162         currentRewards = earned(account);
163
164         // Read withdrawal info in a single storage access
165         PendingWithdrawal storage withdrawal = pendingWithdrawals[account];
166         pendingWithdrawalAmount = withdrawal.amount;
167         withdrawalUnlockTime = withdrawal.unlockTime;
168
169         // Calculate claimable status and remaining lock time
170         isWithdrawalClaimable = (pendingWithdrawalAmount > 0) && (block.timestamp >=
                withdrawalUnlockTime);
171
172         // Calculate remaining lock time
173         if (pendingWithdrawalAmount == 0  block.timestamp >= withdrawalUnlockTime) {
174             remainingLockTime = 0;
175         } else {
176             remainingLockTime = withdrawalUnlockTime - block.timestamp;
177         }
178
179         // Calculate total lifetime rewards
180         totalLifetimeRewards = totalRewardsEarned[account] + currentRewards;
181
182         return (
183             stakedTokens,
184             currentRewards,
185             pendingWithdrawalAmount,
186             withdrawalUnlockTime,
187             isWithdrawalClaimable,
188             totalLifetimeRewards,
189             remainingLockTime
190         );
191     }
```

Listing 3.1: `LIQDStaking::getAccountInfo()`

Recommendation Simplify the above routine to avoid re-compute whether the withdrawal can now be claimed.

Status The issue has been confirmed and will be resolved in a near future commit by following the above suggestion.

3.2 Revisited _syncReward() Logic in LIQDStaking

- ID: PVE-002
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: `LIQDStaking`
- Category: Coding Practices [2]
- CWE subcategory: CWE-1126 [1]

Description

As mentioned earlier, `LIQDStaking` is a liquid staking solution that enables `LIQD` token holders to stake their assets while earning `HYPE` rewards proportional to their stake amount. With that, there is a need to timely update their rewards upon each user interaction. While examining the reward update logic, we notice current implementation can be improved.

To elaborate, we show below the implementation of the related `_syncReward()` function. As the name indicates, this function updates reward state for a given account. Our analysis shows that this function body can be simplified as `rewards[account] = earned(account); userRewardPerTokenPaid[account] = currentRewardPerToken;`. By doing so, we can re-use the same code block (e.g., `earned()`) and enhance the readability and maintenance.

```
667     function _syncReward(address account) internal {
668         // Cache the current reward per token
669         uint256 currentRewardPerToken = rewardRate();
670
671         // Calculate earned rewards
672         uint256 userRewards = rewards[account];
673         uint256 userStaked = stakedAmount[account];
674
675         if (userStaked > 0) {
676             uint256 rewardDelta;
677             unchecked {
678                 // Safe because rewards always increase
679                 rewardDelta =
680                     currentRewardPerToken -
681                     userRewardPerTokenPaid[account];
682             }
683
684             if (rewardDelta > 0) {
685                 uint256 newEarnings = Math.mulDiv(
686                     userStaked,
687                     rewardDelta,
688                     1e18
689                 );
690                 unchecked {
691                     // Safe because we're only adding positive values
692                     userRewards += newEarnings;
```

```
693                 }
694             }
695         }
696
697         // Update user's rewards and checkpoint
698         rewards[account] = userRewards;
699         userRewardPerTokenPaid[account] = currentRewardPerToken;
700     }
```

Listing 3.2: `LIQDStaking::_syncReward()`

Recommendation Revise the above routine with greatly simplified implementation.

Status The issue has been confirmed and will be resolved in a near future commit by following the above suggestion.

PeckShield

4 Conclusion

In this audit, we have analyzed the design and implementation of the LIQD Staking protocol, which is an ERC20 fair launchpad and multi-market DEX aggregator on the HyperEVM. It is designed to allow users to launch tokens and trade them while also returning 100% of revenue to token holders through the LIQD Staking system. Through LIQD Staking, users can safely stake their LIQD tokens and receive proportional emissions of HYPE. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Moreover, we need to emphasize that Solidity-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. https://cwe.mitre.org/data/definitions/1126.html.

[2] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[3] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[4] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[5] PeckShield. PeckShield Inc. https://www.peckshield.com.